Politecnico di Torino

Corso di Laurea Magistrale

A.a. 2025/2026

Programmazione di sistema

Docenti:

Cabodi Gianpiero

Giovanni Malnati

Studente:

Cristiano Berardo

Indice

Elenco delle figure IV

I API Programming 1

1 Architettura e memoria 2
1.1 Obiettivi . . . 2
1.2 Perché studiare come è organizzata la memoria? . . . 2
1.3 Programmi e loro esecuzione . . . 2
1.4 Spazio di indirizzamento . . . 3
1.5 Effetti pratici . . . 3
1.6 Gerarchia di memoria . . . 4
1.7 Località . . . 4
1.8 Esecuzione e programmi di alto livello . . . 4
1.9 Modello di esecuzione di un programma . . . 5
1.10 Livelli di astrazione . . . 5
1.11 Librerie di esecuzione . . . 5
1.12 Modello di esecuzione nei linguaggi C e C++ . . . 6
1.13 Esecuzione sequenziale sincrona, senza limitazioni . . . 6
1.14 Flusso di esecuzione . . . 6
1.15 Stack . . . 6
1.16 Heap . . . 7
1.17 Organizzazione dello spazio di indirizzamento . . . 8
1.18 Ciclo di vita delle variabili . . . 8
1.19 Variabili globali e locali . . . 9
1.20 Variabili dinamiche . . . 9
1.21 Allocazione della memoria . . . 9
1.22 Rilascio della memoria . . . 9
1.23 Puntatori . . . 9
1.24 Il costo del rilascio . . . 10
1.25 Puntatori in C/C++ . . . 10
1.26 Le ambiguità dei puntatori in C/C++ . . . 10
1.27 Problemi legati ai puntatori . . . 10
1.28 Responsabilità del programmatore . . . 11
1.29 Rischi . . . 11
1.30 Problemi dei puntatori in C/C++ . . . 11
1.31 Le risposte di Rust . . . 12
1.32 Gestire i puntatori . . . 12
1.33 Possesso della memoria . . . 12
1.34 Dipendenze . . . 12
1.35 Ma non basta scrivere un programma "giusto"? . . . 13
1.36 Perché questo problema non si verifica in altri linguaggi? . . . 13
1.37 Confronto . . . 13

2 Possesso 14
2.1 Possesso . . . 14
2.2 Movimento . . . 15
2.3 Copia . . . 15
2.3.1 Copia e Movimento . . . 16
2.4 Clonazione . . . 16
2.5 Box<T> - Puntatore con possesso . . . 17
2.6 Riferimenti . . . 18
2.6.1 Riferimenti e prestiti . . . 18
2.6.2 Riferimenti mut - mutual exclusion . . . 18
2.6.3 Riferimenti: disposizione in memoria . . . 19
2.7 Tempo di vita dei riferimenti . . . 19
2.7.1 Esistenza in vita . . . 20
2.8 Riferimenti e funzioni . . . 21
2.9 Riferimenti e strutture dati . . . 22
2.10 Elisione dei tempi di vita . . . 22
2.11 Possesso - riassunto delle regole . . . 23
2.12 Puntatori in Rust . . . 23
2.13 Slice . . . 23
2.14 Vantaggi introdotti dal concetto di possesso . . . 24
2.15 Disposizione in memoria . . . 25

II OS Programming **26**

3 Main memory 27
3.1 Possesso . . . 27
3.2 Movimento . . . 28
3.3 Copia . . . 28
3.3.1 Copia e Movimento . . . 29
3.4 Clonazione . . . 29
3.5 Box<T> - Puntatore con possesso . . . 30
3.6 Riferimenti . . . 31
3.6.1 Riferimenti e prestiti . . . 31
3.6.2 Riferimenti mut - mutual exclusion . . . 31
3.6.3 Riferimenti: disposizione in memoria . . . 32
3.7 Tempo di vita dei riferimenti . . . 32
3.7.1 Esistenza in vita . . . 33
3.8 Riferimenti e funzioni . . . 34
3.9 Riferimenti e strutture dati . . . 35
3.10 Elisione dei tempi di vita . . . 35
3.11 Possesso - riassunto delle regole . . . 36
3.12 Puntatori in Rust . . . 36
3.13 Slice . . . 36
3.14 Vantaggi introdotti dal concetto di possesso . . . 37
3.15 Disposizione in memoria . . . 38

Elenco delle figure

2.1 Single pointer 19
2.2 Fat pointer 19
2.3 Double pointer 19

3.1 Single pointer 32
3.2 Fat pointer 32
3.3 Double pointer 32

Parte I

API Programming

Capitolo 1

Architettura e memoria

1.1 Obiettivi

- Comprendere lo spazio di indirizzamento virtuale
- Conoscere la differenza tra stack e heap
- Saper spiegare perché i puntatori sono pericolosi
- Conoscere quali bug tipici nascono in C/C++
- Spiegare perché Rust introduce i concetti di possesso, prestito, smart pointer

1.2 Perché studiare come è organizzata la memoria?

90% dei bug di sistema → Accessi illeciti alla memoria o mancata sincronizzazione in programmi concorrenti.

Rust offre una risposta a questi problemi

```
int* p = (int*)malloc(sizeof(int));
free(p);
*p = 5; // cosa succede?
```

1.3 Programmi e loro esecuzione

Un programma eseguibile è costituito da un insieme di istruzioni macchina, dati, valori di configurazione e controllo rappresentati come sequenze di numeri binari. Il significato delle istruzioni macchina è cablato all'interno del processore specifico per cui il programma è stato creato. Il significato delle informazioni restanti è contenuto, in parte, nelle istruzioni macchina e, per la parte restante, dipende dal sistema operativo (se c'è) o da particolari caratteristiche del processore stesso.

Per poter essere eseguito, un programma deve essere caricato all'interno della memoria accessibile al processore.

- Nei sistemi più piccoli (es., Arduino) questo avviene grazie ad hardware specifico che memorizza in modo semi-permanente quanto necessario nella memoria flash.
- Nei sistemi più grandi, un modulo del sistema operativo (**loader**) si occupa di trasferire dal disco alla memoria RAM il contenuto del file eseguibile

Una volta che il programma è presente in memoria, può essere eseguito. Il processore preleva dalla memoria, una alla volta, le istruzioni macchina del programma e provvede ad eseguirle.

L'esecuzione di un'istruzione determina quali effetti collaterali debbano avvenire e quale debba essere la prossima istruzione da eseguire. Modifiche nelle informazioni contenute nel processore stesso e/o nella

memoria. Eventuali interazioni con altre periferiche collegate al processore stesso (I/O).

Il modello base di esecuzione prevede l'alternarsi continuo di tre fasi:

- Fetch - Un'istruzione viene trasferita dalla memoria all'interno del processore
- Decode - L'istruzione prelevata viene trasformata in comandi da eseguire
- Execute - I comandi vengono eseguiti

1.4 Spazio di indirizzamento

L'esecuzione di un programma avviene nel suo **spazio di indirizzamento.** Sottoinsieme delle celle indirizzabili, gestito dal sistema operativo. Dati, istruzioni, informazioni di controllo, ... sono memorizzati come valori al suo interno. Lo spazio di indirizzamento può essere immaginato come un enorme array di byte, che possono essere letti individualmente, a coppie, a gruppi di 4, 8 alla volta, in base al parallelismo del processore (8, 16, 32 o 64 bit).

Il numero di celle **potenzialmente** presenti nello spazio di indirizzamento dipende dal processore. Il numero di celle **effettivamente** presenti, è limitato dalla quantità di RAM disponibile ed è controllato dal sistema operativo.

Nel caso di CPU Intel x64, il processore opera a 64 bit. Ma lo spazio di indirizzamento si estende tra 0 e $2^{48} - 1$(scelta di implementazione). All'interno di questo, però, solo una piccola parte di indirizzi è effettivamente presente.

Gli indirizzi che un programma utilizza non corrispondono - in generale - all'effettiva posizione che un dato assume in memoria

- Il codice fa riferimento ad **indirizzi virtuali**
- L'hardware utilizza **indirizzi fisici**

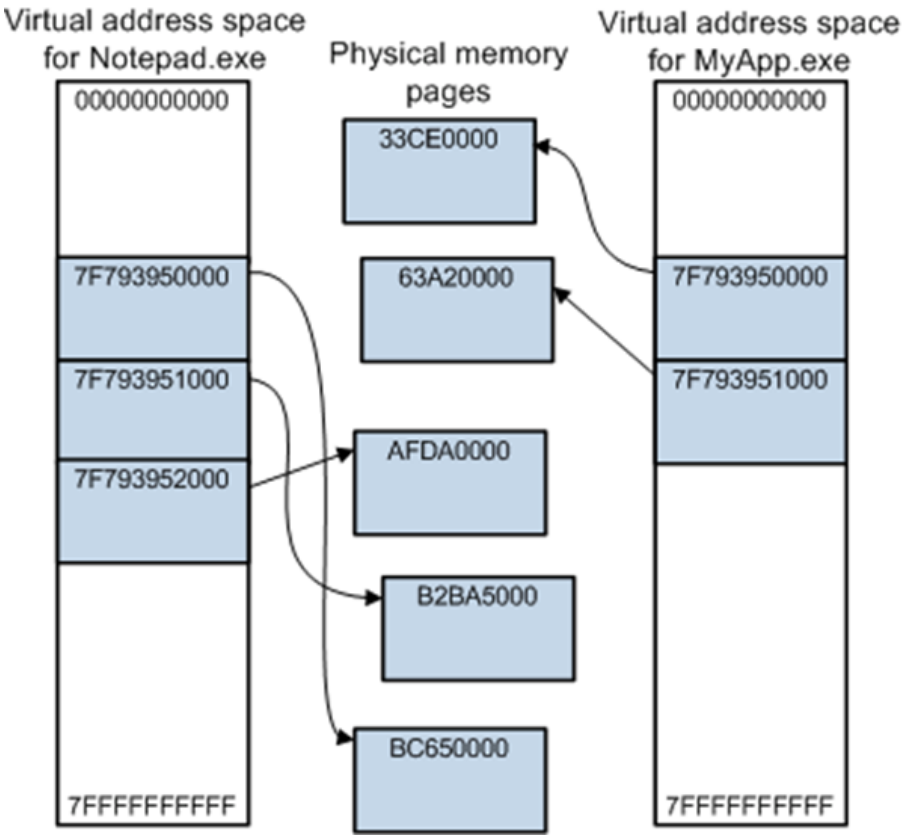
Un apposito blocco hardware presente nel processore (**MMU - Memory Management Unit**) si occupa di tradurre l'indirizzo virtuale in indirizzo fisico.

Tale traduzione garantisce che:

- Programmi diversi contemporaneamente in esecuzione non possano interferire tra loro
- Consente di controllare quali operazioni siano possibili (lettura, scrittura, esecuzione)
- Permette inoltre di scrivere programmi che manipolano dati più grossi della memoria fisica effettivamente presente

1.5 Effetti pratici

- Segmentation fault
  - Tentativo di accedere ad indirizzo non mappato o senza gli adeguati permessi
  - Il processore blocca l'esecuzione e rimanda al sistema operativo (che termina brutalmente il
  - programma)
- Page fault
  - Tentativo di accedere ad un indirizzo appartenente ad un blocco il cui contenuto è stato salvato su disco
  - Il processore richiede l'intervento del sistema operativo che ricerca una pagina fisica libera e vi trasferisce il contenuto del blocco presente nel file di paginazione
  - Al termine il processore ritenta l'accesso e procede normalmente



- Trashing
  - Se il programma cerca di accedere rapidamente ad una grande quantità di dati paginati, il sistema passa più tempo a spostare pagine tra disco e memoria fisica di quanto ne possa dedicare all'esecuzione vera e propria

1.6 Gerarchia di memoria

Nei sistemi non elementari, il processore **non legge/scrive** direttamente dalla memoria RAM. Tra i due elementi viene interposta la memoria **cache**. Questa ha il compito di velocizzare l'accesso alle informazioni cercando di anticipare il bisogno di informazioni da parte della CPU.

Si basa sul **principio di località** dei programmi: se è stato appena fatto accesso all'indirizzo I, è molto probabile che venga fatto subito dopo accesso all'indirizzo $I + \Delta$ (con Δ molto piccolo).

La memoria cache si basa su dispositivi il cui tempo di accesso è molto più rapido di quello offerto dalla memoria RAM tradizionale, ma dotati di una capacità molto inferiore (pochi KB/MB).

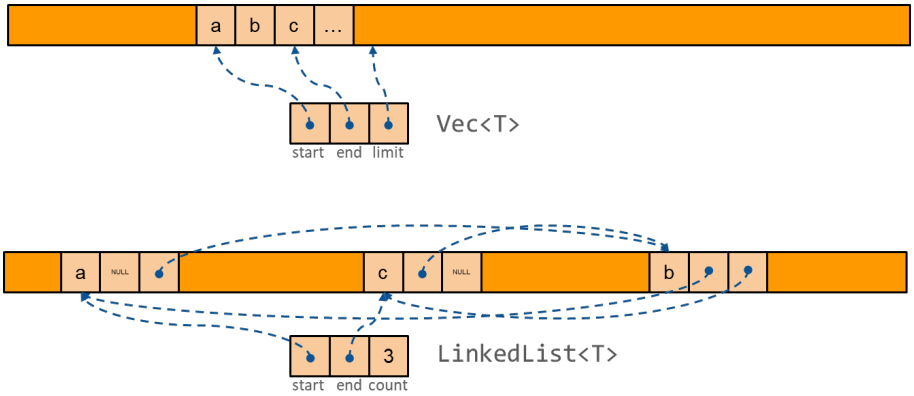
Per questo Vec<T> è quasi sempre meglio di LinkedList<T>

1.7 Località

1.8 Esecuzione e programmi di alto livello

Utilizzando linguaggi di programmazione ad alto livello, i dettagli legati al meccanismo di esecuzione sono normalmente invisibili.

Tuttavia, per poter garantire il livello di astrazione necessario, essi devono introdurre delle sovrastrutture che impattano sul codice che viene scritto dal programmatore... e introducono un insieme di **vincoli** e **restrizioni** di cui è necessario essere pienamente coscienti.



1.9 Modello di esecuzione di un programma

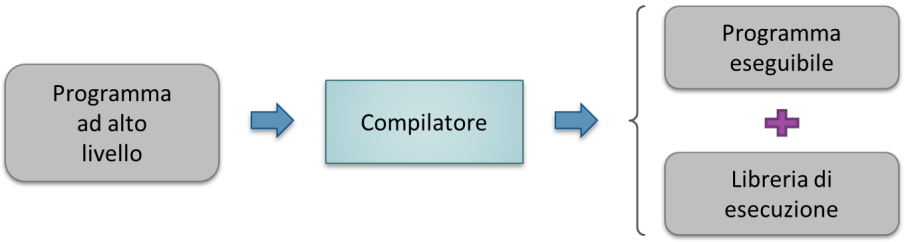
Ogni linguaggio di programmazione propone uno specifico modello di esecuzione, un insieme di comportamenti che attuati dall'elaboratore a fronte dei costrutti di alto livello del linguaggio.

Tale modello per lo più non corrisponde a quello di un dispositivo reale, come fa il processore ad eseguire solo Fatch, Decode ed Execute? Utilizzando strutture dati intermedie e da un insieme di run-time library. È compito del compilatore introdurre uno strato di adattamento che implementi il modello nei termini offerti dal dispositivo soggiacente.

Questo strato è composto, da un lato, da una specifica modalità di traduzione dei costrutti di alto livello in istruzioni macchina e, dall'altro, da un insieme di funzionalità di supporto offerte sotto forma di libreria di supporto all'esecuzione (run-time library).

1.10 Livelli di astrazione

La compilazione trasforma il programma sorgente in un nuovo programma dotato di un modello di esecuzione ed un insieme di istruzioni più semplice. Che può essere eseguito da una macchina fisica (CPU) o subire un'ulteriore trasformazione (esecuzione virtuale)



1.11 Librerie di esecuzione

Offrono agli applicativi meccanismi di base per il loro funzionamento

- Supportano le astrazioni del linguaggio di programmazione
- Forniscono un'interfaccia uniforme tra i diversi S.O. per le funzioni ad essi demandate

Costituite da due tipi di funzioni

- Alcune, invisibili al programmatore, sono inserite in fase di compilazione per supportare l'esecuzione (controllo dello stack, copia di aree di memoria ...)
- Altre offrono al programmatore funzionalità standard, gestendo opportune strutture dati ausiliarie e/o richiedendo al S.O. quelle non altrimenti realizzabili (malloc o fopen)

1.12 Modello di esecuzione nei linguaggi C e C++

In C/C++ sono pensati come se fossero gli unici utilizzatori di un elaboratore, completamente dedicato loro. Le uniche interazioni si riducono al più all'uso di risorse persistenti come il file system o le connessioni di rete .

L'isolamento è reso possibile dal meccanismo di indirizzamento virtuale.

Un programma C/C++ assume di poter accedere a qualsiasi indirizzo di memoria presente nello spazio di indirizzamento, all'interno del quale può leggere o scrivere dati o dal quale può eseguire codice macchina. Ma sappiamo che invece lo spazio di indirizzamento è:

- Limitato
- Non contiguo
- Non tutto lo spazio consente ogni tipo di operazione

1.13 Esecuzione sequenziale sincrona, senza limitazioni

Un programma C/C++ è formato da un insieme di istruzioni eseguite, una per volta, nell'ordine indicato dal programmatore. Non ci sono limiti sul numero di istruzioni da eseguire, sul tempo richiesto alla loro esecuzione, né sulla memoria necessaria .

In realtà, tali limiti esistono e condizionano la scrittura dei programmi. Da qui nasce la necessità di un modello di gestione della memoria sicuro .

All'interno del flusso principale di esecuzione è possibile attivare flussi di elaborazione secondari, detti **thread**. Questi abilitano l'esecuzione concorrente (effettivamente parallela se la CPU è multicore) e introducono un ordine di grandezza nel livello di complessità della scrittura dei programmi. È il S.O. che si occupa di gestire tutti questi flussi di esecuzione indirizzandoli nei vari core fisici.

1.14 Flusso di esecuzione

Il programma è costituito, come minimo, da un flusso di esecuzione il cui punto di partenza è predefinito:

- La funzione main() nel linguaggio C
- I costruttori delle variabili globali in C++, seguiti dalla chiamata alla funzione main()

Vi sono inoltre due strutture dati:

- Lo stack che permette permette di gestire chiamate annidate tra funzioni. Essa **supporta la ricorsione** e l'uso di **variabili locali**. In C++, supporta anche la gestione strutturata delle eccezioni
- Lo heap offre supporto per gestire dinamicamente l'utilizzo della memoria. Secondo logiche non strettamente correlate al procedere dell'esecuzione

1.15 Stack

Blocco di memoria allocato automaticamente all'avvio di un programma, circa 1 MB, al cui interno sarà possibile salvare:

- Variabili locali e temporanee
- Gli argomenti passati ad ogni invocazione di funzione
- Valori di ritorno
- Traccia la storia delle chiamate a funzione

Viene utilizzato a partire da un estremo, espandendosi verso l'estremo opposto ogni volta che avviene una chiamata e contraendosi verso l'inizio quando una funzione ritorna.

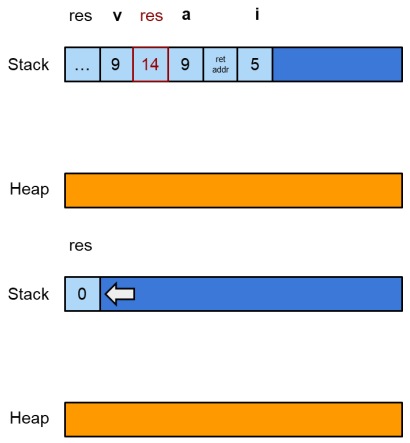
Poiché ha dimensione finita, limita la profondità massima di ricorsione. Così come la dimensione totale delle variabili locali complessivamente utilizzabili.

Se, a seguito di un numero troppo elevato di chiamate o all'allocazione di variabili troppo grandi, si espande oltre il proprio limite, il sistema operativo (o il processore) interviene terminando l'esecuzione del programma generando un Stack overflow.

Nota: Stack = automatico, veloce, tempo di vita deterministico

```
int f(int a) {
    int i = 5;
    if (a>0) {
        return a+i;
    } else {
        return a-1;
    }
}

int main() {
    int v = 9;
    v = f(v);
    return 0;
}
```



L'allocazione ed il rilascio sullo stack sono estremamente efficienti, grazie alla particolare politica di espansione / contrazione adottata.

Per contro, la durata dei valori memorizzati al suo interno è strettamente correlata alla durata dell'invocazione di un funzione. Non è possibile memorizzare nello stack un dato che duri più a lungo della funzione in cui è stato allocato. Per questo motivo, il chiamante di una funzione ha la responsabilità di pre-allocare lo spazio in cui dovrà essere memorizzato il risultato prodotto dalla funzione chiamata.

Inoltre, poiché lo spazio complessivo dello stack è definito a priori, non è possibile memorizzare un dato la cui dimensione superi tale spazio. In generale, occorre limitare la dimensione massima del dato allocato ad una grandezza compatibile con il livello di profondità di chiamate e la dimensione degli altri dati che devono essere memorizzati complessivamente nello stack.

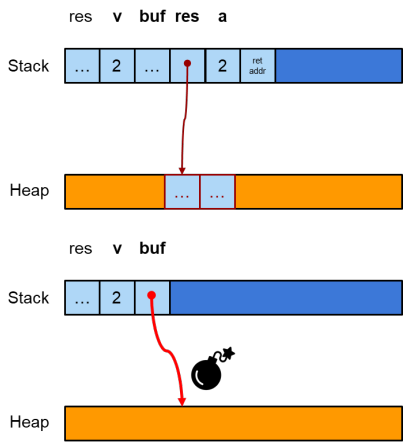
1.16 Heap

Tutte le volte in cui un dato ha un ciclo di vita **non collegato al tempo di esecuzione** della funzione in cui è stato creato o la cui dimensione è **cospicua** oppure **non nota** in fase di compilazione, occorre richiederne la memorizzazione in una struttura a parte. I linguaggi C e C++ offrono un'apposita area, chiamata Heap o Free Store, all'interno della quale è possibile allocare e rilasciare blocchi di dimensione arbitraria (entro il limite della memoria disponibile).

A differenza di quanto avviene per lo stack, le aree allocate sullo heap non sono associate, a livello di linguaggio sorgente, ad un nome di variabile bensì si accede al loro contenuto esclusivamente tramite **puntatori**. Utilizzando l'operatore di dereferenziazione, *, posso accedere al contenuto memorizzato in memoria.

Questo spazio concesso dal S.O. è un prestito, dunque è responsabilità del programmatore rilasciare, prima o poi, la memoria allocata. Altrimenti si verificherà una perdita di memoria (memory leak) che, se reiterata, porta il sistema operativo a distruggere il processo.

```
int* f(int a) {
    if (a>0) {
        return new int[a];
    } else {
        return nullptr;
    }
}
int main() {
    int v = 2;
    int* buf = f(v);     //process buf
    delete[] buf;
    return 0;
}
```



Il puntatore è probabile che sia buono, ma non lo è, non gli è stato assegnato null, il linguaggio non te lo impone.

1.17 Organizzazione dello spazio di indirizzamento

- Codice eseguibile
  - Contiene le istruzioni in codice macchina
  - Accesso in lettura ed esecuzione
- Costanti
  - Accesso in sola lettura
- Variabili globali
  - Accesso lettura/scrittura
- Stack
  - Contiene indirizzi e valori di ritorno, parametri e variabili locali
  - Accesso lettura/scrittura
- Heap
  - Insieme di blocchi di memoria disponibili per l'allocazione dinamica
  - Gestiti tramite funzioni di libreria (malloc, new, free, ...) che li frammentano e ricompattano in base
  - alle richieste del programma

Nel sistema operativo Linux, è facile verificare come sia organizzato lo spazio di indirizzamento di un programma in esecuzione (processo). Ogni processo è identificato univocamente da un numero intero (PID) e con il comando *ps -ef* , il sistema operativo crea, per ciascun processo attivo, un file virtuale denominato */proc/<pid>/maps* che descrive lo spazio di indirizzamento con informazioni sui diversi segmenti al suo interno.

1.18 Ciclo di vita delle variabili

Il modello di esecuzione dei linguaggi C/C++ distingue diverse tipologie di variabili

Globali locali dinamiche

Ciascuna classe ha un proprio ciclo di vita: Intervallo di tempo in cui è garantito l'accesso alle informazioni contenute al loro interno.

1.19 Variabili globali e locali

- Le variabili globali hanno un indirizzo fisso, determinato dal compilatore e dal linker. Sono accessibili in ogni momento e all'avvio del programma, contengono l'eventuale **valore di inizializzazione**.
- Le variabili locali hanno un indirizzo relativo alla cima dello stack. Il ciclo di vita coincidente con quello della funzione/blocco in cui sono dichiarate. Il valore **iniziale è casuale**.

1.20 Variabili dinamiche

Le variabili dinamiche hanno un indirizzo assoluto, determinato in fase di esecuzione. Sono accessibili solo tramite puntatori, è il programmatore a controllarne il ciclo di vita e il valore iniziale può essere inizializzato o meno.

L'uso di questo tipo di variabili presuppone un'infrastruttura di supporto che offra meccanismi di allocazione e di rilascio fornita dalla libreria di esecuzione e dal S.O.

1.21 Allocazione della memoria

La libreria standard C offre vari meccanismi per ottenere l'indirizzo di un blocco allocato dinamicamente

```
void *malloc(size_t s)
void *calloc(int n, size_t s)   //per gli array
void *realloc(void* p, size_t s)   //rialloca aggiungendo memoria
```

In caso di fallimento, viene restituito NULL, tranne nella *realloc(...)* dove il blocco originale non viene toccato e il puntatore *p* resta valido.

In C++ viene definito il costrutto: `new NomeTipo\{argomenti...\}`

1.22 Rilascio della memoria

Opportune funzioni di libreria permettono di restituire i blocchi precedentemente allocati. Poiché ogni funzione di allocazione mantiene le proprie strutture dati, occorre che un blocco sia rilasciato dalla funzione duale di quella con cui è stato allocato:

- `malloc(...) / free(...)`
- `new / delete`
- `new ...[num\_elements] / delete[ ]`

Se il blocco non viene rilasciato si crea una perdita di memoria, che, a lungo andare, provoca l'esaurimento dello spazio di indirizzamento.

Se il blocco viene rilasciato più volte o viene rilasciato con la funzione sbagliata si corrompono le strutture dati degli allocatori, con conseguenze imprevedibili.

1.23 Puntatori

Le variabili sullo heap possono essere gestite solo tramite puntatori perché la loro posizione non è nota a priori.

Vi è un problema, potentissimi per implementare di una vasta gamma di algoritmi come liste, grafo, etc., ma sono estremamente pericolosi. I puntatori sono intrinsecamente ambigui implica che il programmatore gestisca esplicitamente l'allocazione ed il rilascio dei blocchi al loro interno e ne manipoli il contenuto attraverso l'uso dei puntatori.

Il loro utilizzo espone infatti il programmatore ad una vasta gamma di errori, le cui conseguenze portano - nella maggior parte dei casi - a **comportamenti non definiti** (undefined behavior), le cui conseguenze sono disastrose.

1.24 Il costo del rilascio

I linguaggi gestiti, come Java, JavaScript, Python, C#, etc. gestiscono automaticamente il rilascio dei blocchi di memoria utilizzando il Garbage Collector. Per farlo, tengono traccia di chi punta a chi altro. Se, a partire dalle variabili presenti nello stack, un blocco non risulta più raggiungibile, questo può essere liberato.

Per poter applicare questo algoritmo occorre "fermare il mondo". Ovvero **sospendere temporaneamente l'esecuzione del programma intero** per poter osservare una situazione statica e procedere con la compattazione dello heap. Questo può comportare lo spostamento di blocchi e il cambiamento del valore contenuto nei rispettivi puntatori.

L'operazione di rilascio può comportare pause non deterministiche. Talvolta incompatibili con contesti di esecuzione in tempo reale.

1.25 Puntatori in C/C++

Permettono l'accesso diretto ad un blocco di memoria, tale blocco può appartenere ad altri oggetti:

```
int A=10;
int* pA = &A;
```

o essere allocato allo scopo `int* pB = new int{24}`

Possono essere esplicitamente resi invalidi:

- Il valore *0*
- La macro NULL⇒*((void *)0)*
- La parola-chiave *nullptr* (C++11 e superiori)

Quando si usano i puntatori, occorre stabilire quali responsabilità/permessi sono associati al loro uso.

1.26 Le ambiguità dei puntatori in C/C++

- Contiene un indirizzo valido?
- Quanto è grosso il blocco puntato?
- Fino a quando è garantito l'accesso?
- Se ne può modificare il contenuto?
- Occorre rilasciarlo?
- Lo si può rilasciare o altri conoscono lo stesso indirizzo?
- Viene usato come modo per esprimere l'opzionalità del dato?

Nota: In C/C++ il programmatore è responsabile; in Rust la responsabilità è codificata nel sistema dei tipi

1.27 Problemi legati ai puntatori

In C, la gestione della memoria è totalmente affidata al programmatore

- Non ci sono supporti sintattici per automatizzare il ciclo di vita del blocco puntato
- Né meccanismi per associare una specifica semantica al puntatore

In C++ sono state introdotte nuove astrazioni (riferimenti, smart pointer) che sollevano il programmatore da alcune responsabilità, facilitando la creazione di programmi più robusti. Ma richiedono in ogni caso una comprensione approfondita del problema e l'uso attento e coerente dei puntatori.

1.28 Responsabilità del programmatore

- Limitare gli accessi ad un blocco
  - Nello spazio: non accedere alle locazioni che lo precedono o che lo seguono
  - Nel tempo: non accedere al blocco al di fuori del suo ciclo di vita
- Non assegnare a puntatori valori che corrispondono ad indirizzi non mappati
  - Possibile nel caso di cast o di assegnazione improprie
- Rilasciare tutta la memoria dinamica allocata
  - Una e una sola volta, usando la funzione duale di quella usata per la sua allocazione

1.29 Rischi

Accedere ad un indirizzo quando il corrispondente ciclo di vita è terminato, ha **effetti impredicibili**:

- Dangling pointer
- La memoria indirizzata può essere inutilizzata, in uso ad altre parti del programma o non mappata

Non rilasciare la memoria non più in uso, **spreca risorse** del sistema

- Memory leakage
- Se si continua ad allocare senza mai rilasciare, si può saturare lo spazio di indirizzamento

Rilasciare la memoria più volte **corrompe le strutture** usate dallo heap, double free.

Se si assegna ad un puntatore un indirizzo non mappato nello spazio di indirizzamento e si usa quel puntatore, viene generata una interruzione del processore. Il S.O. intercetta tale interruzione e termina il processo.

Se non si inizializza un puntatore e lo si usa, *Wild pointer*, il suo contenuto potrebbe puntare ovunque. Può causare una violazione di accesso oppure corrompere un'area di memoria in uso ad altre parti del programma.

1.30 Problemi dei puntatori in C/C++

Problema	**Conseguenza**
dangling / wild pointer	crash / UB
double free	corruzione heap
leak	esaurimento della memoria
data race	risultati casuali

1.31 Le risposte di Rust

Problema C / C++	Strumento Rust
dangling / wild pointer	borrow checker + tempo di vita
double free	gestione del possesso
leak	pattern RAII
nullptr	tipo Option<T>
gestione del rilascio	tipi Box<T> / RC<T> / Arc<T>

1.32 Gestire i puntatori

Chi alloca un blocco di memoria è **responsabile** di mettere in atto un meccanismo che ne garantisca il successivo **rilascio**. Viene detto "possessore" del blocco.

Cosa capita se si effettua una copia di un puntatore?

Il secondo puntatore si trova coinvolto, suo malgrado, nel ciclo di vita del dato. Occorre introdurre un meccanismo per gestire efficacemente la semantica di un puntatore.

1.33 Possesso della memoria

Il vincolo di rilascio risulta problematico per via dell'**ambiguità** del concetto di puntatore all'interno del linguaggio. Dato un indirizzo non nullo, non è possibile distinguere se sia **valido o meno**, né **a quale area appartenga**, né se occorra o meno **rilasciarlo**.

Ogni volta in cui si alloca un blocco dinamico e se ne salva l'indirizzo in una variabile puntatore, quella variabile diventa proprietaria del blocco: ha la responsabilità di liberarlo.

Non tutti i puntatori posseggono il blocco a cui puntano. Se ad un puntatore viene assegnato l'indirizzo di un'altra variabile, la proprietà è della libreria di esecuzione.

Il problema si complica se un puntatore che possiede il proprio blocco viene copiato. Quale delle due copie è responsabile del rilascio?

1.34 Dipendenze

Quando si introducono strutture dati complesse, è comune che tali strutture debbano appoggiarsi a blocchi di memoria ausiliari in cui memorizzare le informazioni che esse gestiscono.

Un oggetto di tipo `std::vector<T>`, in C++, incapsula un array dinamico di dati di tipo generico T: tali dati possono essere aggiunti, rimossi, sostituiti, ... tramite opportuni metodi sull'oggetto stesso. Tali blocchi ausiliari sono di proprietà dell'oggetto: vengono gestiti dai suoi metodi e devono essere rilasciati quando l'oggetto viene distrutto.

Analogamente, altre strutture dati possono mantenere riferimenti a oggetti del sistema operativo (handle a file, socket, timer, ...).Anche in questo caso le risorse corrispondenti risultano di proprietà dell'oggetto che le gestisce.

Collettivamente, questi tipi di dato prendono il nome di **dipendenze**.

Il linguaggio C non offre nessun supporto sintattico per la gestione delle dipendenze e lascia al programmatore la completa responsabilità della loro gestione

Al contrario, il linguaggio C++ offre, per gli oggetti di tipo *class*, *struct* e *union* (!), la possibilità di specificare particolari funzioni membro dette rispettivamente costruttori e distruttori.

- Un costruttore è una funzione membro che ha il compito di inizializzare un oggetto, provvedendo ad acquisire - se necessario - le sue dipendenze

- Il distruttore è una funzione membro che viene invocata automaticamente dal compilatore quando l'oggetto raggiunge la fine della propria esistenza ed ha il compito di rilasciare correttamente le dipendenze possedute dall'oggetto stesso

1.35 Ma non basta scrivere un programma "giusto"?

Sì, basterebbe se fosse possibile essere certi che lo è! In realtà le cose sono molto più complesse.

Raramente i programmi sono scritti da una singola persona. Per lo più si appoggiano a librerie scritte da altri che hanno fatto opportune assunzioni su come debbano essere utilizzate, ma non è detto che tali assunzioni siano rispettate né, talvolta, conosciute. Spesso si lavora in gruppo e la comunicazione è imperfetta.

Al crescere della dimensione del programma, la quantità di particolari a cui occorre badare cresce molto più in fretta. Ed è estremamente probabile dimenticarsi delle proprie assunzioni.

1.36 Perché questo problema non si verifica in altri linguaggi?

Nella maggior parte dei linguaggi di alto livello il problema della gestione della memoria non si pone, pur offrendo, a vario titolo, il concetto di puntatore.

Java, C#, Python, JavaScript, . . . , basano il proprio meccanismo di rilascio su un algoritmo di Garbage Collection. Libera il programmatore dalla responsabilità di rilasciare esplicitamente la memoria dinamica allocata, in cambio della perdita di controllo su **quando** e **come** tale rilascio avvenga.

La maggior parte di tali linguaggi basa la propria strategia di gestione della memoria su una combinazione dei seguenti algoritmi:

- Reference counting
- Mark and Sweep

1.37 Confronto

Gestione manuale in C/C++
Il rilascio avviene invocando la funzione **free()** o l'operatore **delete**
Il momento in cui avviene il rilascio è controllato dal programmatore
In C++, gli oggetti dispongono di un distruttore che gestisce il rilascio esplicito delle dipendenze
Il rilascio non comporta tempi supplementari di attesa, che sarebbero incompatibili con i sistemi in tempo reale
Si possono verificare memory leak, doppi rilasci, dangling pointer, wild pointer

Capitolo 2

Possesso

2.1 Possesso

In Rust ogni valore introdotto nel programma è **posseduto** da una ed una sola variabile e il suo valore salvato in memoria.

- Il **Borrow Checker** verifica formalmente questo fatto per ogni punto dell'esecuzione del programma
- Ogni violazione porta ad un **fallimento** della compilazione del programma con conseguente errore e spiegazione di quest'ultimo.

Possedere un valore significa essere responsabile del suo **rilascio**.

In particolare se il valore contiene una risorsa quale un puntatore a memoria dinamica, handle di file o socket, etc, questa deve essere liberata. Una volta liberata deve essere restituita al proprietario cui molto spesso è il Sistema Operativo.

Il rilascio avviene quando la variabile che possiede esce dal proprio scope sintattico o quando viene **assegnata** ad un nuovo valore.

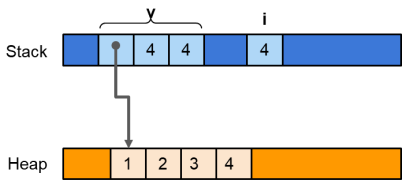
Rust offre un meccanismo di drop mediante il quale è possibile associare azioni arbitrarie da eseguire prima che sia liberata la memoria. Il rilascio può essere rimandato ad un secondo momento se il contenuto della variabile viene trasferito (mosso) in un'altra variabile, la quale diventa responsabile del suo rilascio.

> **Nota:** Questo meccanismo rende l'utilizzo del Garbage collector, presente in linguaggi di programmazione come Java o Python, non necessario.

```rust
fn main() {
    let mut v = Vec::with_capacity(4);
    // v possiede il vec

    for i in 1..=5 {
        v.push(i);
    }

    println!("{:?}",v);
}
```



Inserendo valori all'interno del vettore, lo spazio precedentemente allocato viene via via riempito. Se necessario il blocco viene riallocato per fare spazio ad un maggior numero di elementi.

Finché le variabili sono in scope, i e v, le risorse che posseggono sono accessibili. Una volta usciti dal proprio scope la variabile si occupa dello smaltimento della porzione di stack, ed eventualmente heap, precedentemente allocato seguendo le informazioni contenute in Drop.

2.2 Movimento

Quando una variabile viene inizializzata, essa prende il possesso del relativo valore. Se ad una variabile mutable viene assegnato un nuovo valore, quello precedentemente posseduto viene rilasciato e la variabile diventa proprietaria del nuovo valore.

```
let mut b = a          f(a)          return a
```

In questi tre casi, assegnazione, passaggio ad una funzione e ritorno di una variabile, in Rust, succede qualcosa che in altri linguaggi di programmazione non succede: il valore viene **mosso**, il possesso viene passato dalla variabile originale alla destinazione.

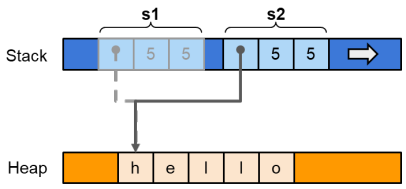
Questo porta ad alcune particolarità:

- La variabile originale cessa di possedere il valore, ovvero non è più responsabile della sua distruzione ed il possesso passa alla variabile di destinazione, o parametro della funzione invocata
- La variabile originale rimane allocata fino a quando non termina la sua visibilità, ovvero al chiusura del blocco in cui è stata definita
- Accessi in **lettura** porteranno ad **errori di compilazione**
- Accessi in **scrittura** alla variabile originale avranno successo e ne riabiliteranno la lettura (si crea un nuovo oggetto)
- La variabile destinazione conterrà una copia bit a bit del valore originale, ammesso che il compilatore non riesca a riusare i dati originali al loro posto

```
let mut s1 = "hello".to_string();
println!("s1: {}", s1);

let s2 = s1;
println!("s2: {}", s2);

//s1 non è più accessibile
...
s1 = "world".to_string();   \\s1 ritorna
↪  accessibile
```



Il movimento in Rust è la soluzione al problema del doppio possesso di parti di aree di memoria. Fisicamente nessuno ha cancellato l'area di memoria ma il compilatore marca $s1$ come inaccessibile.

Il movimento previene l'ambiguità che introduce C/C++: un puntatore è valido/non valido, dobbiamo rilasciare noi la memoria o ci pensa qualcun altro? In Rust è solo il possessore, univoco, della variabile ad occuparsene, i precedenti possessori altro non fanno che contrarre lo stack per liberare lo spazio occupato.

```
let mut s1 = "hello".to_string();
let mut s2 = s1 \\s2 possiede "hello"
...
let mut s1 ...  \\Questo s1 nasconde il precedente s1
```

2.3 Copia

Alcuni tipo, tra cui quelli numerici, sono definiti **copiabili**.

Questa proprietà è definita attraverso il tratto **Copy**. Quando una valore assegnato ad un'altra variabile o usato come argomento in una chiamata a funzione, il valore originale rimane accessibile in **sola lettura**.

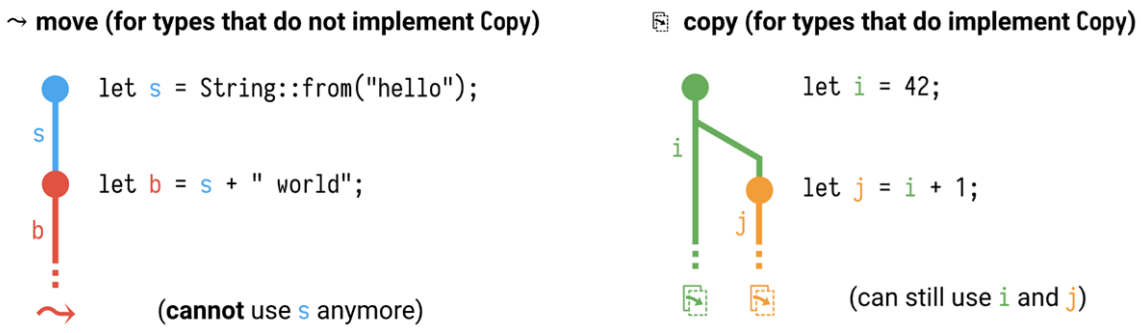
Questo è possibile quando il valore contenuto non costituisce una risorsa che richiede altre azioni di rilascio come ad esempio possono essere File o Socket.

Se *Vec* e *String* posseggono il tratto *Drop*, etichetta per lo smaltimento specifico dato che sono puntatori a zone di heap, *Int*, *Bool*, etc. assumono il tratto *Copy*.

> **Nota:** Drop e Copy sono mutualmente esclusivi. Posso esistere anche tipi né Copy né Drop come per esempio gli oggetti Thread.

I tipo semplici e le loro combinazioni, tuple e array di numeri, sono copiabili. Sono copiabili anche i riferimenti a valori non mutabili.

Viceversa i riferimenti a valori mutabili NON sono copiabili.

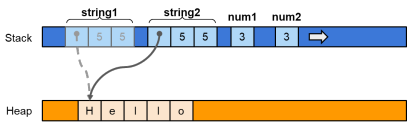


> **Nota:** Dal punto di vista del codice genero, non cambia nulla tra copia e movimento. L'istruzione di assegnazione o passaggio come argomento comporta la duplicazione, bit a bit, del valore originale. In caso di copia il Borrow Checker non impedisce l'ulteriore accesso in sola lettura al dato originale.

2.3.1 Copia e Movimento

Copia e Movimento diventano esattamente la medesima istruzione assembly *mov*, quello che cambia sono le tabelle e le decisioni prese dal compilatore.

```
let string1 = "Hello".to_string();
let string2 = string1;
//da qui in poi, string1 è inaccessibile in
↪  lettura
//a meno che non venga riassegnato
let num1: i32 = 3;
let num2 = num1; //nessun vincolo su num1!
```



2.4 Clonazione

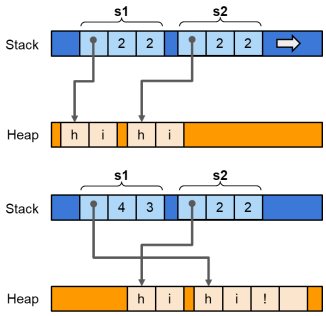
Alcune volte non vogliamo cedere il possesso ma solamente avere una copia e se un tipo implementa il tratto *Clone* può essere duplicato invocando il metodo *clone()*.

A differenza di copia e movimento, la clonazione può implementare una **copia profonda** dei valori. Questo implica un elevato costo di clonazione se il tipo risulta occupare molta memoria.

L'implementazione dell'operazione di clonazione, a differenza di copia e movimento che sono sotto il controllo esclusivo del compilatore ed è basata sull'invocazione della funzione *memcpy(...)*, è modificabile dal programmatore.

Nota: Affinché un tipo possa implementare il tratto Copy, occorre che implementi anche il tratto Clone. Tuttavia, non occorre che le due implementazioni coincidano.

```
let mut s1 = "hi".to_string();
let s2 = s1.clone();
s1.push('!');
println!("s1: {}", s1); //hi!
println!("s2: {}", s2); //hi
```



2.5 Box<T> - Puntatore con possesso

Una variabile locale è SEMPRE allocata sullo stack. Questo si estende di una quantità pari alla dimensione del valore che deve essere memorizzato nella variabile. Quando la variabile esce dal proprio scope sintattico, lo stack si contrae ed il valore viene rilasciato.

In alcune situazioni:

- Non è nota la dimensione a priori
- Occorre prolungare il tempo di vita del valore oltre al blocco sintattico in cui è definito
- La dimensione è maggiore dello stack stesso

l'oggetto deve essere allocato sullo heap, utilizzando il tipo generico **Box<T>**. Una variabile di questo tipo contiene un puntatore al valore.

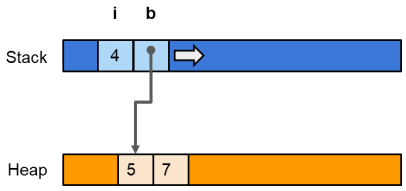
Per allocare un valore con il tipo Box si scrive:

```
let b = Box::new(v);
```

dove v è un qualsiasi valore. L'istruzione precedente definisce una variabile b la quale è un puntatore ad un area di memoria sullo heap contenente il valore v.

Si accede al valore contenuto con l'espressione $*b$[1], e se la variabile è definita come mutabile è possibile modificare il contenuto a cui si punta.

```
fn useBox() {
    let i = 4;
    let mut b = Box::new( (5, 2) );
    (*b).1 = 7;
    println!("{:?}", *b); // (5,7)
}
```

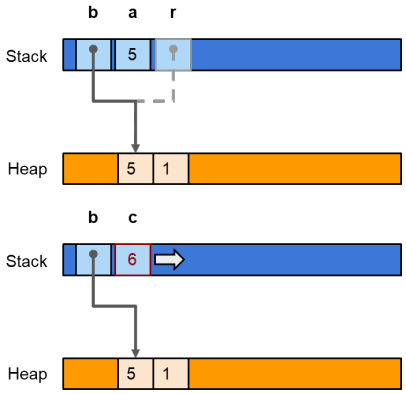


Quando l'esecuzione del programma raggiungerà la fine del blocco di codice in cui la variabile b è stata definita, il blocco verrà rilasciata a patto che il contenuto di b, il puntatore al blocco sia stato **mosso** in un'altra variabile.

[1] Si può scrivere pure *b.1*, il compilatore lo de-referenzia in autonomia (*b).1

```
fn makeBox(a: i32) -> Box<(i32,i32)> {
    let r = Box::new( (a, 1) );
    return r;
}

fn main() {
    let b = makeBox(5);
    let c = b.0 + b.1;
}
```



2.6 Riferimenti

Per risolvere i problemi di accesso, Rust introduce diverse forme di puntatori, volte a rendere espliciti responsabilità e diritti di chi le maneggia. Il borrow checker si occupa di garantire che il codice sia scritto in conformità con tali regole e impedisce la compilazione in caso contrario.

Un **riferimento** è un puntatore in sola lettura ad un blocco di memoria **posseduto da un'altra variabile**.

- Permette di accedere ad un valore senza trasferirne la proprietà
- Il riferimento può esistere solo mentre esiste la variabile che possiede il dato
- non serve utilizzare *, quando si accede al valore tramite l'operatore .

```
let point = (1.0, 0.0); //point possiede il valore
let reference = &point; //reference PUÒ accedere al valore in lettura
//finché esiste point
println!("({}, {})", reference.0, reference.1);
```

2.6.1 Riferimenti e prestiti

Un riferimento prende in prestito, borrows, in sola lettura, l'indirizzo di memoria in cui esiste il valore.

Fino a che il riferimento è accessibile, non è possibile modificare il valore, né tramite il riferimento, il quale è in sola lettura, né tramite la variabile che possiede il valore.

È possibile creare ulteriori riferimenti a partire dal dato originale o da altri riferimenti ad esso.

- I riferimenti sono **copiabili**: viene duplicato il puntatore
- Il compilatore, attraverso il BC, garantisce che un riferimento punti sempre ad un dato valido
- Finché esiste almeno un riferimento, il dato originale **non potrà essere né modificato né distrutto**

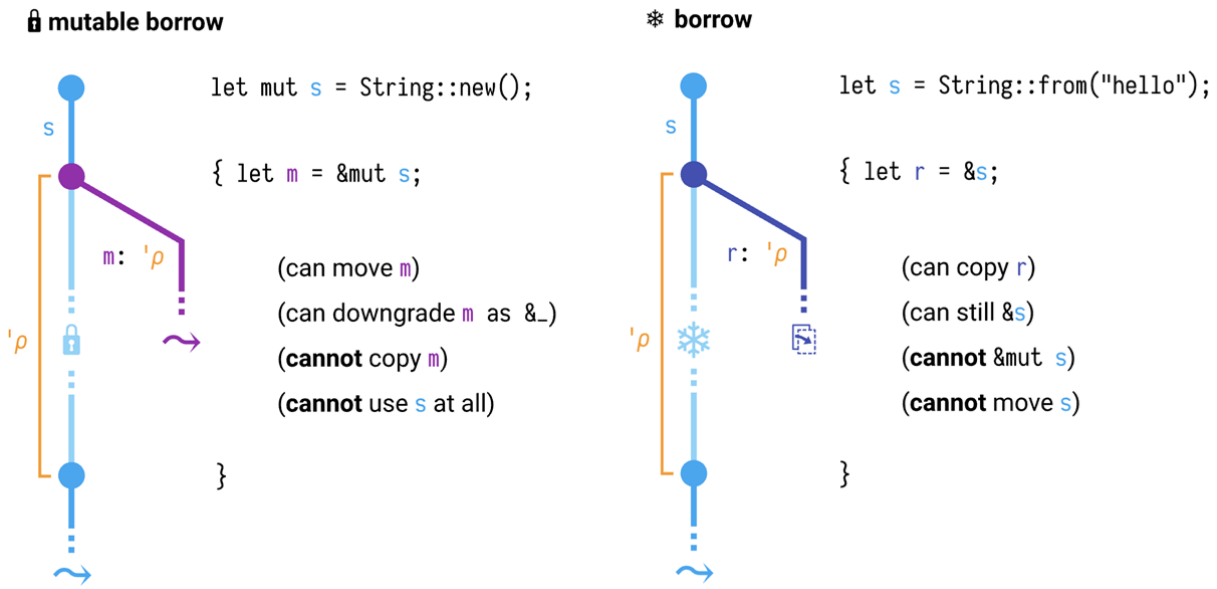
2.6.2 Riferimenti mut - mutual exclusion

A partire da una variabile che possiede un valore è possibile estrarre **un solo riferimento mutabile** per volta. `let r = &mut v;`.

Mentre esiste un riferimento mutabile

- Non possono esistere riferimenti semplici, in sola lettura
- Non è possibile modificare né muovere la variabile che possiede il valore

È possibile creare un riferimento mutabile solo se la variabile che possiede il dato è a sua volta dichiarata come mutabile.



2.6.3 Riferimenti: disposizione in memoria

I riferimenti sono implementati diversamente in base al tipo puntato

- Puntatori semplici, se il compilatore conosce la dimensione del dato puntato
- Puntatore + dimensione (fat pointer), se la dimensione del dato è solo nota in fase di esecuzione. Nelle stringhe tolgo il problema di inserire il terminatore.
- Puntatore doppio, se il tipo di dato puntato è noto solo per l'insieme di tratti che implementa. Esempi File o Socket.

```
let i: i32 = 10;
let r1: &i32 = &i;
```

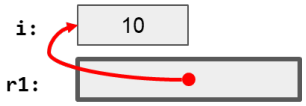


Figura 2.1: Single pointer

```
let a: [i32;3] = [1,2,3];
let r2: &[i32] = &a;
```

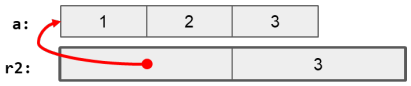


Figura 2.2: Fat pointer

```
let f: File = File::create("test.txt");
let r3: &dyn Write = &f;
```

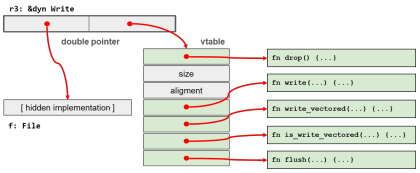


Figura 2.3: Double pointer

2.7 Tempo di vita dei riferimenti

Il BC garantisce che tutti gli accessi ad un riferimento avvengano solo in un intervallo di tempo, o righe, compreso in quello in cui il dato esiste. Serve ad impedire il fenomeno dei **dangling pointer**.

L'insieme delle righe in cui si fa accesso al riferimento costituisce il suo **lifetime**. Tale info è mantenuta dal compilatore assieme ad altre informazioni, quale il tipo, e non ha alcuna rappresentazione in fase di esecuzione.

Sebbene il tempo di vita in molte situazioni possa essere omesso, il tempo di vita di un riferimento può essere espresso nella firma del tipo. Questo aiuta sia il programmatore sia il compilatore per il mantenimento dell'informazione.

- `&'a NomeTipo`, dove *a* è un identificativo qualsiasi e il simbolo ' serve a quantificarlo come tempo di vita
- Tale notazione è utile in quelle situazioni in cui occorre imporre vincoli sulla durata relativa dei tempo di vita di due o più riferimenti

Se un riferimento deve durare l'intera durata del programma viene indicato con `&'static NomeTipo`.

> **Nota:** Una stringa espressa in formato letterale ("Some string") ha come tipo `&'static str`, in quanto la sequenza di caratteri viene allocata dal compilatore nella sezione delle costanti e non viene mai rilasciata.

Per essere lecito, il tempo di vita di un riferimento deve essere contenuto, minore, nel tempo di vita del valore a cui punta. Sempre il BC si fa carico di garantire il vincolo.
Questo richiede di esplicitare il tempo di vita quando si memorizza un riferimento all'interno di una struttura dati, come per esempio una tupla, o si usa un riferimento come valore di ritorno di una funzione.

Il vincolo (apparentemente ovvio) di esistenza in vita dei vincoli dovrebbe essere alla base anche di tutti gli usi dei puntatori in C e C++.
Poiché, però, in questi linguaggi non viene tracciato il tempo di vita, il compilatore non è in grado di identificare la presenza di dangling pointers, né identificare eventuali problemi di non rilascio o doppio rilascio, come invece avviene in Rust.

```
{
    let r;
    {
        let x = 1;
        r = &x;
    }
    assert_eq!(*r, 1);
}

//error: `x` does not live long enough
```

2.7.1 Esistenza in vita

Le regole, ovviamente, valgono anche quando si crea un riferimento ad una parte di una struttura dati più grande.
L'esistenza in vita del riferimento deve essere inclusa in quella della struttura a cui punta

```
let v = vec![1, 2, 3];
let r = &v[1]; // v deve durare più a lungo di r
```

Se, al contrario si memorizzano dei riferimenti in una struttura, tutti questi riferimenti devono avere una durata di vita maggiore della struttura dati in cui sono memorizzati.

```
{
    let mut v = Vec::new();
    {
```

```
        let a = 1;
        v.push(&a);
    }
    println!("{:?}", v);
}

\\error[E0597]: `a` does not live long enough
```

2.8 Riferimenti e funzioni

Se una funzione riceve un parametro di tipo riferimento (mutabile o meno), il tempo di vita del riferimento diventa parte integrante della firma della funzione È necessario infatti che tutte le operazioni eseguite dalla funzione sul riferimento siano compatibili con la validità del dato memorizzato al suo interno

`fn f(p: &i32) {...}` ⇒ `fn f<'a>(p: &'a i32) {...}`

Il compilatore provvede in molti casi a effettuare autonomamente la riscrittura indicata (lifetime elision).

Se la funzione riceve più riferimenti, può essere necessario indicare se il loro tempo di vita sia vincolato al più breve o se siano disgiunti

- Nel primo caso si usa un solo identificatore `fn f<'a>(p1: &'a i32, p2:&'a i32) {...}`
- Nel secondo si usano etichette diverse `fn f<'a, 'b>(p1: &'a i32, p2:&'b i32) {...}`

Uno dei casi più frequenti in cui il compilatore non riesce a dedurre il corretto tempo di vita è legato a funzioni che restituiscono un riferimento, estratto da una delle strutture dati ricevute in ingresso.
In tale caso occorre annotare il tipo restituito con la corretta etichetta

```
fn f<'a, 'b>(p1: &'a Foo, p2:&'b Bar) -> &'b i32 { /*...*/ return &p2.y; }
```

Se la funzione memorizza il riferimento ricevuto in ingresso in una struttura dati, il compilatore deduce che il tempo di vita della struttura in cui il riferimento è memorizzato deve essere incluso o coincidente con il tempo di vita del riferimento.
Se questo non avviene, il compilatore identifica l'errore e impedisce alla compilazione di avere successo.

```
fn f(s: &str, v: &mut Vec<&str>) {
    v.push(s);  //error: lifetime mismatch
}

fn f<'a>(s: &'a str, v: &'a mut Vec<&str>) {
    v.push(s);
}
//error: explicit lifetime required in the type of `v`

fn f<'a>(s: &'a str, v: &'a mut Vec<&'a str>) {
    v.push(s);
}

fn main() {
    let mut v: Vec<&str> = Vec::new();
    {
        let s= String::from("abc");
        v.push(&s);     //error: `s` does not live long enough
    }
    println!("{:?}",v);
}
```

Se una funzione ha due o più riferimenti in ingresso e un riferimento in uscita, il compilatore non è in grado di decidere (senza eseguire il codice) da quale parametro provenga il riferimento in uscita. L'inserimento degli identificatori nella firma della funzione risolve questa ambiguità.

```
fn f<'b, 'c>(x: &'b S, y: &'c S) -> &'c u8 {...}

let v1 = S(1);
let mut v2 = S(2);

let r = f(&v1, &v2);     // Sappiamo che r è basato su v2,
                         // che rimane nello stato di prestito
                         // mentre v1 viene rilasciato e può evolvere
v2 = v1; // Questa operazione crea un problema!
print_byte(r); // Qui finisce il prestito di r (e di v2)
```

Lo scopo degli identificatori relativi al ciclo di vita è duplice:

- Per il codice che invoca la funzione (chiamante), essi indicano su quale, tra gli indirizzi in ingresso, è basato il risultato in uscita
- Per il codice all'interno della funzione (chiamato), essi garantiscono che vengano restituiti solo indirizzi cui è lecito accedere per (almeno) il tempo di vita indicato

All'atto dell'invocazione, gli identificatori forniti dal programmatore (o inseriti automaticamente dal compilatore, quando possibile) sono legati all'effettivo intervallo minimo (espresso come insieme di linee di codice) nel quale il valore da cui il prestito è stato preso debba restare bloccato per non violare le assunzioni su cui la funzione è basata.

Tentativi di modificare il valore originale da cui il prestito è preso prima che il tempo di vita sia trascorso portano ad errori di compilazione che costituiscono la base della robustezza del sistema di possesso e prestito offerto da Rust

2.9 Riferimenti e strutture dati

Lo stesso tipo di ragionamento vale se il dato viene salvato all'interno di una qualsiasi struttura dati

- Rust verifica che il valore a cui si punta abbia un tempo di vita maggiore o uguale del tempo di vita della struttura dati
- Questo richiede di esplicitare il tempo di vita della struttura rispetto al tempo di vita dei riferimenti in essa contenuti

Se una struttura che contiene riferimenti è contenuta, a sua volta, in un'altra struttura, anche quest'ultima deve avere il tempo di vita specificato in modo esplicito.

```
struct User<'a> {
    id: u32,
    name: &'a str,
}
```

```
struct Data<'a> {
    user: User<'a>,
    password: String,
}
```

2.10 Elisione dei tempi di vita

Rust, per default, provvede ad assegnare, ad ogni riferimento presente in una struttura dati o tra i parametri di una funzione, un tempo di vita distinto. Tali tempi di vita si propagano al codice che fa uso di tale struttura dati e, in assenza di ambiguità, non richiede di esplicitare gli identificatori dei tempi di vita.

```
struct Point {
    x: &i32,
    y: &i32,
}

fun scale(r: &i32, p: Point) -> i32 {
    r * (p.x * p.x + p.y * p.y)
}
```

```
struct Point {
    x: &'a i32,
    y: &'b i32,
}
fun scale<'a,'b,'c> (r: &'a i32, p: Point<'b,
↪   'c>) -> i32 {
    r * (p.x * p.x + p.y * p.y)
}
```

Se una funzione restituisce un riferimento o un tipo che contiene - direttamente o indirettamente - un riferimento, occorre disambiguare il tempo di vita del valore ritornato

- Se c'è un solo parametro in ingresso dotato di tempo di vita, Rust assume che quello debba essere il tempo di vita del risultato
- Se sono presenti più parametri in ingresso con tempo di vita, tocca al programmatore esplicitare quale debba essere il tempo di vita da associare al risultato

Nel caso di metodi che accedono a self tramite un riferimento, Rust assume che il tempo di vita da associare al risultato sia quello del riferimento a self.

Questo parte dal presupposto che se un metodo di un oggetto restituisce un dato preso a prestito (borrow), questo sia stato preso dai dati posseduti dall'oggetto stesso.

2.11 Possesso - riassunto delle regole

- Ciascun valore ha un unico possessore (variabile o campo di una struttura). Il valore viene rilasciato (drop) quando il possessore esce dal proprio scope o quando al possessore viene assegnato un nuovo valore.
- Può esistere, al più, un singolo riferimento mutabile ad un dato valore
- **Oppure**, possono esistere molti riferimenti immutabili al medesimo valore. Ma fintanto che ne esiste almeno uno, il valore non può essere mutato.
- Tutti i riferimenti devono avere una durata di vita inferiore a quella del valore a cui fanno riferimento

2.12 Puntatori in Rust

Proprietà	**&T**	**Box<T>**
Possesso	No: prestito	Possessore unico
Rilascio	A carico del possessore	Automatico al termine dello scope
Validità	Garantita dal compilatore	Garantita in fase di esecuzione
Nullabilità	Mai nullo	Mai nullo
Aliasing e mutabilità	Controllata: o molti riferimenti in sola lettura o un solo riferimento mutabile	Soggetta alla regole del possesso (no aliasing)

2.13 Slice

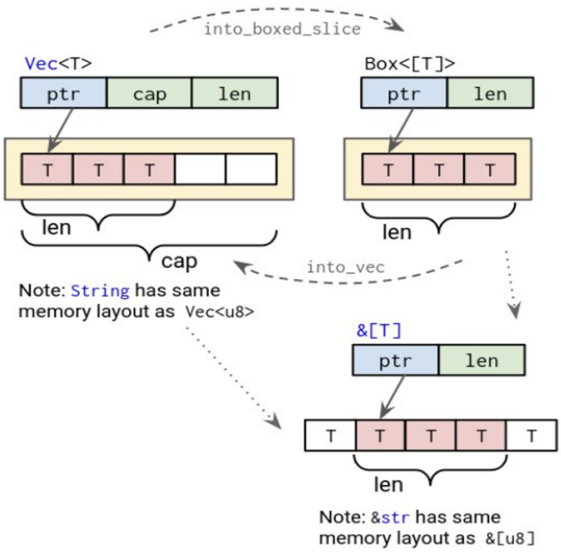
Una slice (fetta) è una VISTA di una sequenza contigua di elementi, la cui lunghezza NON è nota in fase di compilazione, ma disponibile durante l'esecuzione.

Internamente viene rappresentata come una tupla di due elementi, il primo dei quali punta al primo valore della sequenza, mentre il secondo indica il numero di elementi consecutivi. Una slice di elementi T ha tipo [T].

Una slice **non possiede** i dati cui fa riferimento. Questi appartengono sempre ad un'altra variabile, da cui la slice viene derivata. Tutti i valori contenuti sono garantiti essere inizializzati, gli accessi sono verificati in fase di esecuzione.

È possibile ricavare una slice a partire da un array, ma anche da altri tipi di contenitori quali std::Vec<T>, String, Box<[T]>, etc.

```
let a: [i32; 5] = [1, 2, 3, 4, 5]; // a è un array di 5 interi
let s = &a[1..3]; // s è una slice formata da 2 elementi [2,3]
let two = s[0]; // two contiene il valore 2
```



2.14 Vantaggi introdotti dal concetto di possesso

Un programma Rust, pur non avendo un garbage collector, offre molteplici garanzie relative alla correttezza degli accessi in memoria e del rilascio delle risorse.

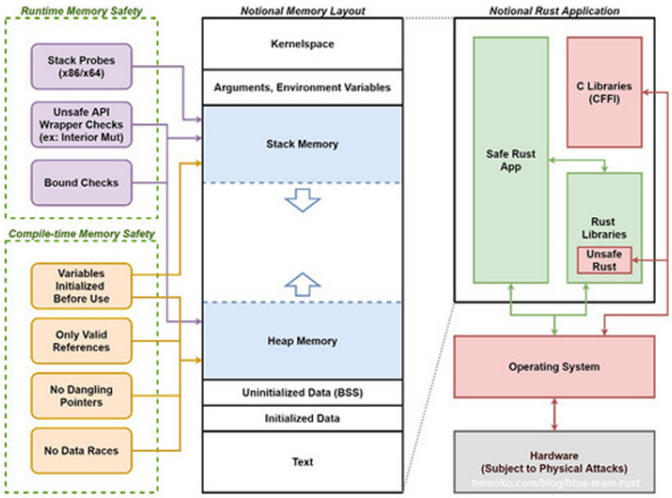
- Non esiste il concetto di **riferimento nullo** e, di conseguenza, non c'è il rischio di dereferenziarlo
- Poiché il borrow checker vigila sulle assegnazioni dei riferimenti e sugli intervalli temporali in cui essi sono effettivamente usati impedendo accessi illeciti, non c'è il rischio di originare errori di segmentazione o accesso illegale ad aree ristrette di memoria, né la possibilità di avere riferimenti ad aree già rilasciate (dangling pointer)
- Poiché in ogni istante il borrow checker è in grado di determinare la dimensione di un blocco di memoria cui un riferimento punta, non possono verificarsi **buffer overflow** né **buffer underflow**
- Per lo stesso motivo, gli iteratori offerti da Rust **non eccedono** mai i loro limiti

Tutte le variabili sono **immutabili** per default e occorre una dichiarazione esplicita per renderle mutabili. Questo obbliga il programmatore a riflettere attentamente sul come e dove i dati debbano essere modificati e su quale sia il ciclo di vita di ciascun valore.

Il modello di possesso non riguarda solo la gestione della memoria, ma anche la **gestione delle risorse** contenute in un valore come socket di rete, handle di file e database, descrittori dei dispositivi.

L'assenza di un garbage collector impedisce **comportamenti non deterministici**. In particolare, la sospensione totale del funzionamento ogni qual volta occorra ricompattare la memoria.

2.15 Disposizione in memoria



Parte II

OS Programming

Capitolo 3

Main memory

3.1 Memoria

Processo: eseguibile in esecuzione

3.2 Address binding

Chi scrive il sw non vuole vedere l'indizzzo di memoria, vogliamo utilizzare indirzzi simboloci e solo il compilatore poi li trasformerà in indirizzi relativi.

Il linker decide di riallocare a valori assoluti.

3.3 Binding di instruzoni e dati in memoria

- compile time: Codice assoluto, si se lo sai già, altrimetni ricolabile. Sto generando l'eseguibile.
- Load time: Fare il binding se gli inidirzzi non sono ancora decisi, generare un indirizzo assoluto o rilocabile e usato in fase di esecuzione.
- Execution time: Facciamo il binding sono quando il codice viene eseguito fino a che il processo viene mandato in esecuzione ed eventualmente spostato. La CPU al minimo deve offrire il Base register + limit register, deve dare supporto.

3.4 Memory Managment Unit - MMU

Ho bisogno di un metodo per passare dalla memoria virtuale alla memoria hardware e viceversa, il processo di binding.

Hanno senso quando sono diversi in fase di esecuzione.

Address space porzione di memoria in cui un programma può utilizzare

3.4.1 Logico

3.4.2 Fisico

3.5 MMU

La MMU è all'interno della CPU. Se si legge l'assembler leggiamo indizzi logici.

Ci sono diversi tipi di MMU dipende da chi hai assunto e da quando vuoi pagare un chip.

3.5.1 Relocatin register

Se abbiamo una MMU basta su Relocation register, gli ind logici sono bound al volo, quando hai ritatrsato il binding all'esecuzione. è un sommatore + un comparatore per bloccare se usi ind troppo gandi.

Ma non tutto funziona così. Dynamic loading e dynamic loading

3.5.2 dynamic loading

il pgm non è detto che può stare tutto in memoria. e non è detto che lo voglio tutto subito in memoria. perché? spero una parte o tutto non mi serva più (è fatta in sequenza cencetto di località soazione e temprale).

Non solo, 10K istruzioni, e parti completamente non utilizzate! ho deciso di sottoutilizzare il pgm oppure per righe di codice per errori etc

perché non carichiamo solo quando una routine viene chiamata?

Poi ci consenza di non caricare le cose che non ci servo, ma il loader deve intervenire qunado serve e il file deve essere possibile caricarlo in momenti diversi(vedi il formato) Utile quanndo il codice è grande e quando statisticamente alcune parti non venfono utilizzate.

Per quanto riguarda il linking, cosa significa? Il linker fa statico o dimatico?

3.5.3 static linking

Le lib se sono statiche le agganci tutte. Difetto eseguibili grossi.

3.5.4 dynamic linking

Non fai tutto i link, ti tieni degli indizizzi non risolti (eg non sai la pintf etc) quando li saprai in fase di load o fase di esecuzione.

cosa c'è al posto di printf, si mette un indizzo ad una routine , stoub, che fa finta di essere la routne e una volta arrivata li viene rimpizzata con la printf vera.

Utile per le librerie! è utile per le shared librery, una sola copia in memoria e tutti la vedono: tutti la trovano già caricata MA spazio di indirizzamento condiviso!! Problema del versioning delle liberie shared, normalmente basta aggiornare.

3.6 Excutable file

è un file binario e non di testo, con 1 header, codice e dati e sa symbol table

Linking: prndi un insieme di pgm (file oggetto) e crea un file caricabile (image) nel fare questo risolvi i simboli Loading: copiare l'eseguibile in memoria e magari agganciarlo con pezzo già caricate (eg shared library) e aggiorarre gli indizzi se sevono.

3.7 Classic unix

Unix usa gcc. Il linker vive dentro gcc il loader è parte di una system call Exec static loading

3.8 Dynaimc loading

3.9 Programm-controlled dynamic loading

Il programma si gestisce il load e sostituisce lui. Serve solo la LOAD e non la printf! Eg la printf nella prima chiamata carica la printf e tutte le altre volte invece già presa dalla memoria.

3.9.1 linker assidted dynamic loading

Lo stab myPrintf fa subito la load e in qualche modo il loader/linker verrà poi sostituita in esecuzione, l'indirizzo cambia e c'è più inerzione con il sistema operativo. Cmabia la responsabilità di chi fa qualcosa

3.10 shared library

Alcuni pgm usano la stessa fuznone, quindi facciamo un load automatico e il linker non la risolve. se il loader è già in memoria la carico altrimenti lo eseguito

3.11 dynamic linking

Il linking non è fatto in memoria completo e verrà fatto quello completo a livello di esecuzione.

Solo dire caricamenot dinamico degli indizii un po' alla volta. il dynamic loading significa solo che il programma non è caricato tutto in una botta ma un po' alla volta.

3.12 Dynamic shared library

sono le dll che usano sia dynamic loading e linking. Che sono l'altro apsetto della medaglia diq aundo abbiamo deciso che ci sono delgi indizii fisici e logici.

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because LaTeX now knows how many pages to expect for this document.